Universidad Mariano Gálvez de Guatemala
Centro Universitario Cede de Jalapa
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación
Ciclo: Quinto
Catedrático: Ingeniero Marco Tulio Valdez Albizurez
Curso: Programacion III

Trabajo: investigación Task en C# y Paralelización

Estudiante:	Carnet:
Christopher Ricardo García Girón	0907-24-10087

Jalapa 15 de mayo de 2026

¿Qué son los Task y Threads en C#?

Threads:

Un Thread es la unidad básica de ejecución dentro de un proceso. Un programa puede tener distintos hilos ejecutándose de manera simultanea

En el momento que una aplicación inicia, automáticamente crea un hilo principal llamado Main Thread el cual es el encargado de ejecutar el método Main

Los threads nos permiten

- Ejecutar procesos en segundo plano
- Realizar distintas operaciones simultáneamente
- Evitar que la interfaz gráfica se congele
- Distribuir tareas pesados en distintos hilos.

Cada thread tiene:

- Su propio flujo de ejecución
- Su pila de memoria
- Su propio tiempo de procesamiento el cual es asignado por el sistema operativo.

Task:

Las task son las que representan operaciones asíncronas y son una forma moderna de trabajar con concurrencia en .NET

- Sirven para simplificar el manejo de los hilos
- Utilizan internamente el Thread Pool
- Son eficientes para crear Threads manualmente
- Facilitan programación asíncrona
- Se deben integrar con async y await

Thread pool

Es un conjunto de hilos reutilizables que son administrados por .NET

En vez de crear hilos nuevos constantemente el sistema reutiliza hilos existentes mejorando el rendimiento y reduciendo el consumo de recursos.

¿Cómo se usan los Task?

Se utilizan para ejecutar procesos en segundo plano sin bloquear el programa principal.

- Debemos crear una task
- Seguido esperar la misma task

¿Para que sirven las palabras reservadas async y await?

Async:

nos indica que un método puede contener operaciones asíncronas

Await:

pausa la ejecución del método hasta que la tarea termine, pero sin bloquear el hilo

Funcionamiento:

- El método se pausa temporalmente
- El hilo queda libre
- La tarea continua en segundo plano
- Al finalizar el método continuo donde quedo

Ventajas del Async y await

- Interfaces más fluidas
- Evitan congelamientos
- Código más fácil de leer
- Mejor experiencia de usuario

paralelización de tareas

La paralelización consiste en dividir u trabajo en varias partes para ejecutarlas simultáneamente lo cual nos permite aprovechar varios núcleos del procesador.

- Tenemos el:
- parallel.for
- El parallel.ForEach
- Task.WhenAll: este nos permite esperar múltiples tareas simultáneamente.

Ventajas de la paralelización:

- Mayor velocidad
- Aprovechamiento multinúcleo
- Reducción de tiempos de procesamiento.

Desventajas

- Mayor complejidad
- Problemas de sincronización
- Posibles conflictos entre hilos

Colecciones Concurrentes

Las colecciones concurrentes permiten que varios hilos accedan simultáneamente a estructuras de datos sin generar errores

Problemas con las colecciones normales

Las colecciones tradicionales como list o dictionary no son seguras para múltiples hilos

Si varios hilos modifican una colección al mismo tiempo pueden ocurrir

- Datos corruptos
- Excepciones
- Resultados incorrectos

Tipos de colecciones concurrentes

CurrentQueue = Cola FIFO segura

ConcurrentStack = Pila LIFO segura

ConcurrentDictionary = Diccionario concurrente

BlockingCollection = Productor-consumidor

Ventajas de las colecciones concurrentes

- Seguridad entre hilos
- Mejor rendimiento
- Evitan bloqueos manuales

Referencias

learn.microsoft. (21 de 10 de 2025). Obtenido de https://learn.microsoft.com/es-es/dotnet/standard/collections/thread-safe/

learn.microsoft. (25 de 10 de 2025). Obtenido de https://learn.microsoft.com/es-es/dotnet/api/system.threading.thread?view=netframework-4.8.1&

learn.microsoft. (18 de 8 de 2025). Obtenido de https://learn.microsoft.com/es-es/dotnet/csharp/asynchronous-programming/async-scenarios

learn.microsoft. (22 de 10 de 2025). Obtenido de https://learn.microsoft.com/es-es/dotnet/standard/parallel-programming/task-based-asynchronous-programming

Learn.microsoft. (25 de 10 de 2025). Obtenido de https://learn.microsoft.com/es-es/dotnet/standard/parallel-programming/